

Sprawozdanie z Listy 5 (Laboratorium)

Obliczenia Naukowe

Agnieszka Głuszkiewicz

1 Zadanie: Układ równań z macierzą blokową

1.1 Opis problemu

Celem zadania było rozwiązanie układu równań liniowych $Ax = b$, gdzie $b \in \mathbb{R}^n$, a $A \in \mathbb{R}^{n \times n}$ jest macierzą rzadką o specyficznej strukturze blokowej. Macierz A zdefiniowana jest następująco:

$$A = \begin{pmatrix} A_1 & C_1 & 0 & 0 & 0 & \cdots & 0 \\ B_2 & A_2 & C_2 & 0 & 0 & \cdots & 0 \\ 0 & B_3 & A_3 & C_3 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & B_{v-2} & A_{v-2} & C_{v-2} & 0 \\ 0 & \cdots & 0 & 0 & B_{v-1} & A_{v-1} & C_{v-1} \\ 0 & \cdots & 0 & 0 & 0 & B_v & A_v \end{pmatrix} \quad (1)$$

gdzie n jest podzielne przez l , $v = n/l$ to liczba bloków, a $l \geq 2$ to rozmiar bloków wewnętrznych $A_k, B_k, C_k \in \mathbb{R}^{l \times l}$. Charakterystyka bloków:

- A_k – macierz gęsta
- B_k – macierz posiadająca niezerowe elementy tylko w pierwszym wierszu i ostatniej kolumnie
- C_k – macierz diagonalna

Zadanie wymagało implementacji algorytmu eliminacji Gaussa o złożoności obliczeniowej $O(n)$ (dla stałego l), uwzględniającego rzadkość macierzy, w dwóch wariantach: bez wyboru elementu głównego oraz z częściowym wyborem elementu głównego.

1.2 Rozwiązanie

1.2.1 Algorytm eliminacji Gaussa dla macierzy blokowej

Standardowa eliminacja Gaussa posiada złożoność $O(n^3)$. Ze względu na specyficzną strukturę macierzy (niezerowe elementy występują tylko w wąskim pasmie wokół przekątnej), możliwe jest ograniczenie operacji tylko do elementów niezerowych.

Dla każdego kroku eliminacji k , operacje odejmowania wierszy wykonywane są tylko dla wierszy i w zakresie od $k + 1$ do $limit$, gdzie $limit$ to zasięg niezerowych elementów w kolumnie k wynikający ze struktury bloków A_k i B_{k+1} . Zasięg ten ograniczony jest przez rozmiar dwóch bloków ($2l$).

Algorithm 1 Eliminacja Gaussa dla struktury blokowej

Require: Macierz A , wektor b , rozmiar n , rozmiar bloku l

Ensure: Wektor rozwiązań x

```
1: for  $k \leftarrow 1$  to  $n - 1$  do
2:    $limit \leftarrow \min(n, k + 2l - (k \bmod l))$ 
3:   for  $i \leftarrow k + 1$  to  $limit$  do
4:      $multiplier \leftarrow A[i, k] / A[k, k]$ 
5:     for  $j \leftarrow k$  to  $limit$  do
6:        $A[i, j] \leftarrow A[i, j] - multiplier \cdot A[k, j]$ 
7:     end for
8:      $b[i] \leftarrow b[i] - multiplier \cdot b[k]$ 
9:   end for
10: end for
11:  $x[n] \leftarrow b[n] / A[n, n]$ 
12: for  $k \leftarrow n - 1$  down to 1 do
13:    $sum \leftarrow 0$ 
14:    $limit \leftarrow \min(n, k + 2l - (k \bmod l))$ 
15:   for  $j \leftarrow k + 1$  to  $limit$  do
16:      $sum \leftarrow sum + A[k, j] \cdot x[j]$ 
17:   end for
18:    $x[k] \leftarrow (b[k] - sum) / A[k, k]$ 
19: end for
20: return  $x$ 
```

1.2.2 Wybór elementu głównego

Aby uniknąć błędów numerycznych spowodowanych dzieleniem przez liczbę bliską zera, zaimplementowano wariant z częściowym wyborem elementu głównego. W k -tym kroku poszukiwany jest element o największym module w kolumnie k , ale poszukiwania ograniczone są do zakresu, w którym występują elementy niezerowe (wewnątrz bloku A_k i B_{k+1}).

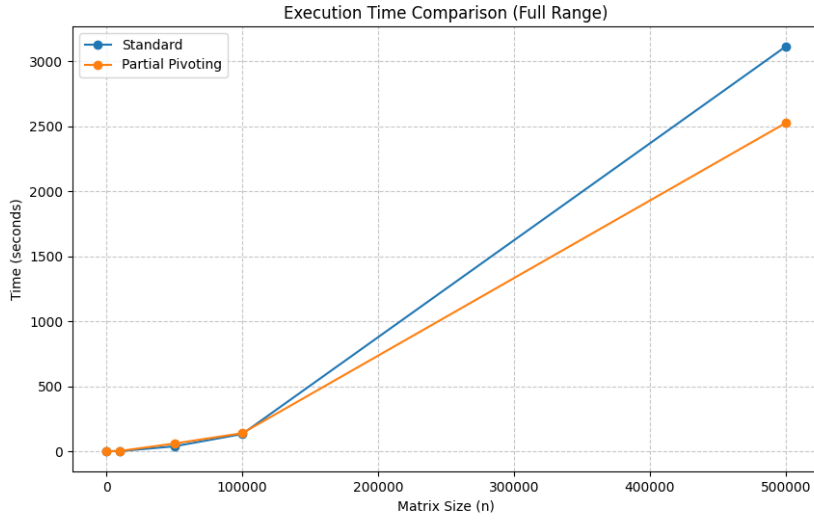
1.2.3 Złożoność obliczeniowa i pamięciowa

- **Złożoność obliczeniowa:** W każdej pętli zewnętrznej (wykonywanej n razy) liczba operacji w pętlach wewnętrznych zależy od stałej l (rzędu $4l^2$), a nie od n . Dla stałego l , złożoność wynosi $O(n)$.
- **Złożoność pamięciowa:** Macierz przechowywana jest w strukturze rzadkiej (SparseMatrixCSC o Julii). Liczba niezerowych elementów rośnie liniowo z n , zatem zużycie pamięci wynosi $O(n)$.

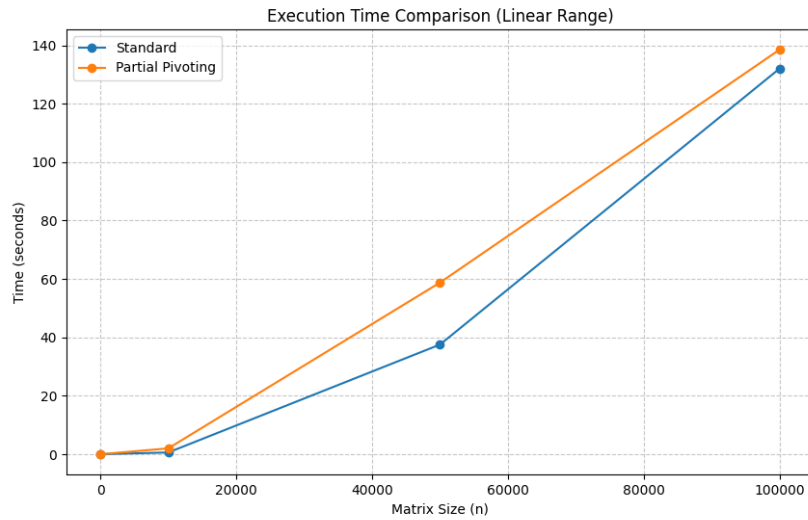
1.3 Wyniki eksperymentów

1.3.1 Czas działania algorytmów

Poniższe wykresy przedstawiają czasy wykonania algorytmu dla różnych rozmiarów macierzy n . W implementacji wykorzystano macierze rzadkie (SparseMatrix).



Rysunek 1: Porównanie czasu działania algorytmów dla większego zakresu danych



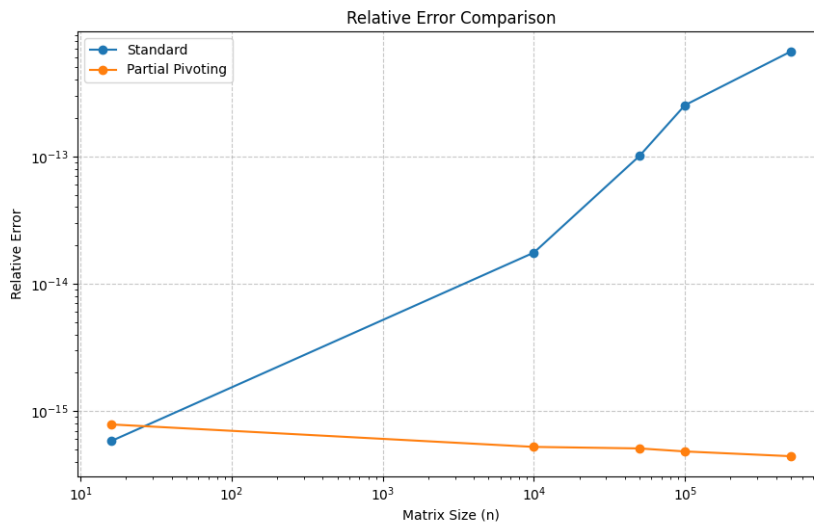
Rysunek 2: Porównanie czasu działania algorytmów dla mniejszego zakresu danych

Pierwszy wykres wskazuje, że dla dużych wartości n ($n = 500,000$) wzrost czasu staje się nieliniowy. Wynika to prawdopodobnie z powodu użycia struktury `SparseMatrixCSC`, która jest zoptymalizowana pod operacje kolumnowe, a eliminacja Gaussa wymaga z kolei wielu operacji na wierszach. Oprócz tego zakładamy, że dostęp do wartości w strukturze jest rzędu $O(1)$, co niekoniecznie musi być prawdą.

Jednak, jak widać na drugim wykresie (z wyłączeniem skrajnego przypadku $n = 500,000$), dla wyników do $n = 100,000$ zależność czasu od rozmiaru problemu jest bliska liniowej.

1.3.2 Błąd względny rozwiązań

Analizowano dokładność rozwiązania poprzez wyznaczenie błędu względnego $\frac{\|x_{num} - x_{dok}\|}{\|x_{dok}\|}$, gdzie $x_{dok} = (1, \dots, 1)^T$.



Rysunek 3: Błąd względny algorytmów w skali logarytmicznej.

Wykres błędów potwierdza przewagę algorytmu z wyborem elementu głównego pod kątem dokładności wyniku. Linia pomarańczowa (Partial Pivoting) jest płaska i utrzymuje się na poziomie bliskim precyzji maszynowej, niezależnie od rozmiaru macierzy. Natomiast dla algorytmu standardowego (linia niebieska) obserwujemy wzrost błędu wraz ze wzrostem n .

Jednak obserwowane wartości błędów względnych niezależnie od metody osiągają niewielkie wartości dzięki poprawnemu uwarunkowaniu macierzy.

1.4 Wnioski

1. Algorytm z częściowym wyborem elementu głównego wykazał się dobrą stabilnością. Błąd względny pozostał na poziomie epsilon maszynowego nawet dla dużych wartości n . Przy algorytmie bez pivotingu można zauważyć, że błędy rosną wraz z rozmiarem macierzy. Niezależnie od metody osiągnięte wartości błędów są jednak niewielkie ze względu na dobre uwarunkowanie macierzy.
2. Przez użycie struktury `SparseMatrixCSC` czas obliczeń czasem rośnie szybciej niż liniowo, co prawdopodobnie wynika z jej optymalizacji pod kątem kolumnowym, a nie wierszowym oraz niewystarczające informacje co do czasu dostępu do danych w tak zapisanej macierzy.
3. Zaimplementowane rozwiązanie całkiem poprawnie rozwiązuje problem układów blokowych, ale aby utrzymać lepszą wydajność dla dużych danych, konieczna byłaby rezygnacja z `SparseArrays` na rzecz innej, bardziej optymalnej struktury danych.